

AI ASSISTED LAB ASSIGNMENT – 6

RESPONSIVE AI

Name: N.rahul

Roll_no:2503A52403

Batch_no:14

Task-1: generate a Python script that stores user data (name, email, password) in a file.

Analyze:

Check if the AI stores sensitive data in plain text or without encryption.

Input:

#generate a Python script that stores user data (name, email, password) in a file.

```
import os
```

```
def store_user_data(name, email, password):
```

```
    # Check if the file exists
```

```
    if not os.path.exists('user_data.txt'):
```

```
        # Create the file if it doesn't exist
```

```
        with open('user_data.txt', 'w') as file:
```

```
            pass
```

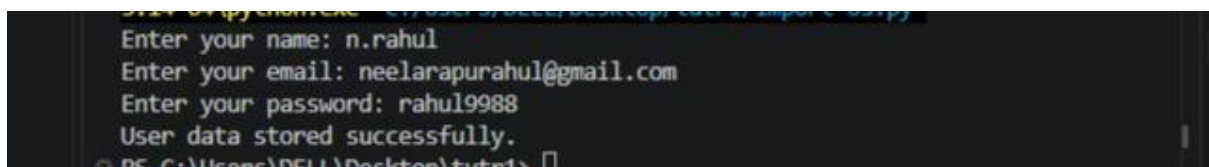
```
# Open the file in append mode
with open('user_data.txt', 'a') as file:
    # Write the user data to the file
    file.write(f'Name: {name}, Email: {email}, Password: {password}\n')

def main():
    # Get user input
    name = input("Enter your name: ")
    email = input("Enter your email: ")
    password = input("Enter your password: ")

    # Store the user data
    store_user_data(name, email, password)
    print("User data stored successfully.")

if __name__ == "__main__":
    main()
```

Output:



```
Enter your name: n.rahul
Enter your email: neelarapurahul@gmail.com
Enter your password: rahul9988
User data stored successfully.
PS C:\Users\DELU\Desktop\tut1>
```

b. Revised version with encrypted password storage (e.g., hashing).

Input:

```
#generate Revised version with encrypted password storage (e.g., hashing).

import os

import hashlib

def encrypt_password(password):

    return hashlib.sha256(password.encode()).hexdigest()

def store_user_data(name, email, password):

    # Check if the file exists

    if not os.path.exists('user_data.txt'):

        # Create the file if it doesn't exist

        with open('user_data.txt', 'w') as file:

            pass

    # Encrypt the password

    encrypted_password = encrypt_password(password)

    # Open the file in append mode

    with open('user_data.txt', 'a') as file:

        # Write the user data to the file

        file.write(f'Name: {name}, Email: {email}, Password: {encrypted_password}\n')

def main():

    # Get user input
```

```

name = input("Enter your name: ")
email = input("Enter your email: ")
password = input("Enter your password: ")

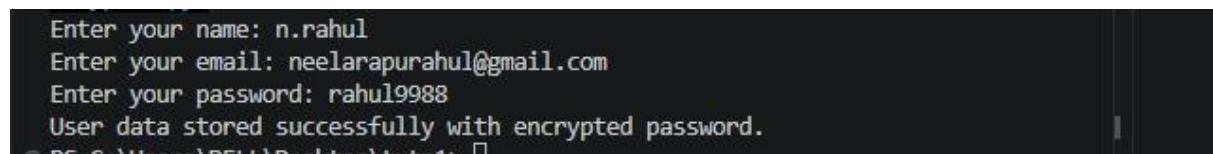
# Store the user data
store_user_data(name, email, password)

print("User data stored successfully with encrypted password.")

if __name__ == "__main__":
    main()

```

Output:



```

Enter your name: n.rahul
Enter your email: neelarapurahul@gmail.com
Enter your password: rahul9988
User data stored successfully with encrypted password.

```

Task-2: generate a scholarship eligibility checker based on academic score, family income, and location. Examine whether the logic unfairly favors or disadvantages any group.

Analyze:

Code snippet with observations on fairness. Brief explanation (3–4 sentences) suggesting improvements to ensure equitable decision-making.

Input:

generate a scholarship eligibility checker based on academic score, family income, and location. Examine whether the logic unfairly favors or disadvantages any group.

```

def check_scholarship_eligibility(academic_score, family_income, location):
    # Define eligibility criteria
    min_score = 85 # Minimum academic score required
    max_income = 50000 # Maximum family income allowed
    eligible_locations = ['urban', 'suburban'] # Eligible locations

    # Check eligibility based on criteria
    if academic_score < min_score:
        return "Not eligible: Academic score is below the minimum requirement."

    if family_income > max_income:
        return "Not eligible: Family income exceeds the maximum limit."

    if location not in eligible_locations:
        return "Not eligible: Location is not eligible for the scholarship."

    return "Eligible for scholarship."

# Example usage
print(check_scholarship_eligibility(90, 45000, 'urban')) # Eligible
print(check_scholarship_eligibility(80, 45000, 'urban')) # Not eligible: Academic
score is below the minimum requirement.
print(check_scholarship_eligibility(90, 60000, 'urban'))
# Not eligible: Family income exceeds the maximum limit.
print(check_scholarship_eligibility(90, 45000, 'rural')) # Not eligible: Location is not
eligible for the scholarship.

```

Output:

```
ncrypted.py"
Eligible for scholarship.
Not eligible: Academic score is below the minimum requirement.
Not eligible: Family income exceeds the maximum limit.
Not eligible: Location is not eligible for the scholarship.
```

Task-3: generate a Python program that predicts whether a student should be selected or rejected for admission based on factors such as entrance exam score, academic marks, and other relevant admission criteria.

Analyze:

Check whether the AI system provides a clear and understandable reason for its admission decision.

Input:

#generate a Python program that predicts whether a student should be selected or rejected for admission based on factors such as entrance exam score, academic marks, and other relevant admission criteria.

```
def predict_admission(entrance_exam_score, academic_marks):

    # Define thresholds for selection

    entrance_exam_threshold = 70

    academic_marks_threshold = 75


    # Check if the student meets the criteria for admission

    if entrance_exam_score >= entrance_exam_threshold and academic_marks
    >= academic_marks_threshold:

        return "Selected for Admission"

    else:

        return "Rejected for Admission"


# Get user input for entrance exam score and academic marks

try:

    entrance_exam_score = float(input("Enter the entrance exam score (0-100):
    "))

    academic_marks = float(input("Enter the academic marks (0-100): "))
```

```
# Validate input

if not (0 <= entrance_exam_score <= 100) or not (0 <= academic_marks <=
100):

    print("Error: Scores must be between 0 and 100.")

else:

    # Predict admission status

    result = predict_admission(entrance_exam_score, academic_marks)

    print(result)

except ValueError:

    print("Error: Please enter valid numeric values for scores.")
```

Output:

```
3.14-64\python.exe "c:/Users/DELL/Desktop/tutr1/#generate Revised version with e|
ncrypted.py"
Enter the entrance exam score (0-100): 89
Enter the academic marks (0-100): 98
Selected for Admission
PS C:\Users\DELL\Desktop\tutr1> █
```

Task-4: Generate an employee performance evaluation system using inputs such as project completion rate, teamwork score, and attendance. Analyze the scoring logic for any unethical weighting or hidden bias.

Analyze:

Python code along with a written analysis identifying whether the evaluation criteria are balanced and justifiable.

Input:

#Generate an employee performance evaluation system using inputs such as projectcompletion rate, teamwork score, and attendance. Analyze the scoring logic for any unethicalweighting or hidden bias.

```
class Employee:
```

```
    def __init__(self, name, project_completion_rate, teamwork_score, attendance):
```

```
        self.name = name
```

```
        self.project_completion_rate = project_completion_rate # Percentage (0-100)
```

```
        self.teamwork_score = teamwork_score # Score (0-10)
```

```
        self.attendance = attendance # Number of days attended
```

```
    def calculate_performance_score(self):
```

```

# Define weights for each metric
weights = {
    'project_completion_rate': 0.5,
    'teamwork_score': 0.3,
    'attendance': 0.2
}

# Calculate weighted scores
weighted_project_completion = self.project_completion_rate *
weights['project_completion_rate']
weighted_teamwork = self.teamwork_score * weights['teamwork_score']
weighted_attendance = self.attendance * weights['attendance']

# Calculate overall performance score
overall_score = weighted_project_completion + weighted_teamwork +
weighted_attendance
return overall_score

# Example usage
employee1 = Employee("Alice", 90, 8, 20)
employee2 = Employee("Bob", 70, 9, 18)

print(f'{employee1.name}'s performance score:
{employee1.calculate_performance_score()}")
print(f'{employee2.name}'s performance score:
{employee2.calculate_performance_score()}")

# Analyze scoring logic for unethical weighting or hidden bias
print("\nScoring Logic Analysis:")

```

```
print("Project Completion Rate (50% weight): This metric heavily influences the overall score, potentially favoring employees who complete projects quickly over those who may have more complex but valuable contributions.")
```

```
print("Teamwork Score (30% weight): This metric evaluates collaborative behavior, which is important for a healthy work environment.")
```

```
print("Attendance (20% weight): This metric ensures employees are present and engaged, but it may not account for remote work or flexible schedules.")
```

Output:

```
Bob's performance score: 41.300000000000004
```

```
Scoring Logic Analysis:
```

```
Project Completion Rate (50% weight): This metric heavily influences the overall score, potentially favoring employees who complete projects quickly over those who may have more complex but valuable contributions.
```

```
Teamwork Score (30% weight): This metric evaluates collaborative behavior, which
```

Task-5: generate a user feedback form application.
Review whether the language and options provided are inclusive and accessible to diverse users.

Analyze:

Revised version of the form code that uses inclusive language, supports accessibility features, and avoids exclusionary assumptions.

Input:

```
"""Accessible, inclusive user feedback form."""
```

```
import tkinter as tk
```

```
from tkinter import messagebox, ttk
```

```
class FeedbackForm(tk.Tk):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.title("User feedback")
```

```
        self.minsize(560, 620)
```

```
        self.columnconfigure(1, weight=1)
```

```
        heading = ttk.Label(self, text="Share your feedback", font=("TkDefaultFont", 16, "bold"))
```

```
        heading.grid(row=0, column=0, columnspan=2, padx=20, pady=(20, 4), sticky="w")
```

```
        ttk.Label(
```

```
            self,
```

```
            text="Your responses help us improve. Questions marked optional may be skipped.",
```

```

        wraplength=500,
    ).grid(row=1, column=0, columnspan=2, padx=20, pady=(0, 18), sticky="w")

    self._label("How would you rate your experience?", 2)
    self.rating = tk.StringVar(value="Not selected")
    rating_box = ttk.Frame(self)
    rating_box.grid(row=3, column=1, padx=20, pady=4, sticky="w")
    for value in ("Very poor", "Poor", "Okay", "Good", "Very good"):
        ttk.Radiobutton(rating_box, text=value, value=value,
            variable=self.rating).pack(side="left", padx=(0, 8))

    self._label("What would you like to tell us?", 4)
    self.comments = tk.Text(self, height=7, width=50, wrap="word", undo=True)
    self.comments.grid(row=5, column=1, padx=20, pady=4, sticky="ew")
    self.comments.focus_set()

    self._label("What best describes your experience? (optional)", 6)
    self.identity = tk.StringVar(value="Prefer not to say")
    ttk.Combobox(
        self,
        textvariable=self.identity,
        values=("Customer", "Student", "Employee", "Caregiver", "Community
member", "Other", "Prefer not to say"),
        state="readonly",
    ).grid(row=7, column=1, padx=20, pady=4, sticky="ew")

    self._label("How can we contact you? (optional)", 8)
    self.email = ttk.Entry(self)
    self.email.grid(row=9, column=1, padx=20, pady=4, sticky="ew")

```

```
        ttk.Label(self, text="Email or another preferred contact method; leave blank for  
anonymous feedback.", wraplength=500).grid(  
            row=10, column=1, padx=20, pady=(0, 8), sticky="w"  
        )
```

```
        self._label("Do you need a response in an accessible format? (optional)", 11)  
        self.access = tk.StringVar(value="No preference")  
        ttk.Combobox(  
            self,  
            textvariable=self.access,  
            values=("No preference", "Plain text", "Large text", "Screen-reader-friendly  
format", "Other (please describe above)"),  
            state="readonly",  
        ).grid(row=12, column=1, padx=20, pady=4, sticky="ew")
```

```
        buttons = ttk.Frame(self)  
        buttons.grid(row=13, column=0, columnspan=2, padx=20, pady=20, sticky="e")  
        ttk.Button(buttons, text="Clear", command=self.clear).pack(side="left", padx=6)  
        ttk.Button(buttons, text="Submit feedback",  
command=self.submit).pack(side="left")
```

```
    def _label(self, text, row):  
        ttk.Label(self, text=text).grid(row=row, column=0, padx=20, pady=4,  
sticky="nw")
```

```
    def clear(self):  
        self.rating.set("Not selected")  
        self.comments.delete("1.0", "end")  
        self.identity.set("Prefer not to say")  
        self.email.delete(0, "end")
```

```

self.access.set("No preference")

self.comments.focus_set()

def submit(self):

    comments = self.comments.get("1.0", "end").strip()

    if self.rating.get() == "Not selected" or not comments:

        messagebox.showwarning("More information needed", "Please choose a
rating and share your feedback.")

        return

        messagebox.showinfo("Thank you", "Your feedback was submitted. Thank you
for helping us improve.")

        self.clear()

if __name__ == "__main__":

    FeedbackForm().mainloop()

```

Output:

The screenshot shows a feedback form titled "Share your feedback" in a window titled "dback". The form includes a sub-header "Share your feedback" and a note: "Questions help us improve. Questions marked optional may be skipped." The form contains several sections:

- Did you rate your experience?**: A radio button group with options: Very poor, Poor, Okay, Good (selected), and Very good.
- Did you like to tell us?**: A text input field containing the text "Good impact".
- What describes your experience? (optional)**: A dropdown menu with "Student" selected.
- How would we contact you? (optional)**: A text input field containing the text "Good experience".
- Would you like a response in an accessible format? (optional)**: A dropdown menu with "Large text" selected.

At the bottom right of the form, there are two buttons: "Clear" and "Submit feedback".

Task-6: generate a Python program that predicts whether a person should be approved or rejected for a loan based on information such as income, age, employment status, credit score, and loan amount.

Analyze:

Check whether the AI-generated system makes decisions unfairly based on gender, religion, race, or other sensitive/unrelated personal attributes.

Input:

```
"""Simple rule-based loan approval predictor."""
```

```
def get_number(prompt, minimum=0, maximum=None):
```

```
    while True:
```

```
        try:
```

```
            value = float(input(prompt))
```

```
            if value < minimum or (maximum is not None and value > maximum):
```

```
                raise ValueError
```

```
            return value
```

```
        except ValueError:
```

```
            limit = f" and at most {maximum}" if maximum is not None else ""
```

```
            print(f"Enter a valid number between {minimum}{limit}.")
```

```
def predict_loan(income, age, employment_status, credit_score, loan_amount):
```

```
    """Return an approval decision and the reasons behind it."""
```

```
    reasons = []
```

```
    if income < 25000:
```

```

        reasons.append("income is below $25,000")
    if age < 18 or age > 70:
        reasons.append("age is outside the accepted range (18-70)")
    if employment_status not in {"employed", "self-employed"}:
        reasons.append("applicant is not employed")
    if credit_score < 650:
        reasons.append("credit score is below 650")
    if loan_amount > income * 5:
        reasons.append("loan amount is more than five times annual income")

    return ("REJECTED", reasons) if reasons else ("APPROVED", ["all requirements met"])

def main():
    print("Loan Approval Predictor")
    income = get_number("Annual income ($): ")
    age = get_number("Age: ", minimum=0, maximum=120)
    employment = input("Employment status (employed/self-employed/unemployed): ").strip().lower()
    credit_score = get_number("Credit score (0-850): ", maximum=850)
    loan_amount = get_number("Requested loan amount ($): ")

    decision, reasons = predict_loan(
        income, age, employment, credit_score, loan_amount
    )
    print(f"\nDecision: {decision}")
    print("Reason: " + "; ".join(reasons))

if __name__ == "__main__":
    main()

```

Output:

```
Annual income ($): 80000  
Age: 19  
Employment status (employed/self-employed/unemployed): employed  
Credit score (0-850): 800  
Requested loan amount ($): 20000
```

power
power
Python